ITC - DS 2

Durée: 2h **2025-2026** **13/12/2025**

Préparatif du Père Noël

L'utilisation de toute calculatrice, de tout matériel électronique et de tout document est interdite

- On considérera comme acquise toute fonction demandée à une question précédente, même si celle-ci n'a pas été traitée. Il est donc toujours possible de faire les dernières questions même si les questions précédentes n'ont pas été traitées. De manière générale, on recommande de faire appel aux fonctions codées au fil du problème.
- Tout programme écrit au crayons de papier sera ignoré.
- L'indentation doit être claire.
- Ici on vous donne les signatures des fonctions avec les types, lorsque vous codez la fonction vous n'avez pas besoin de mettre les types.

1. Gestion du stock de jouets

Depuis le début de son activité en 1855, le père Noël gère son stock de jouets avec une liste de chaînes de caractères répertoriant tous les jouets de son entrepôt.

Par exemple, avec `liste_1` cela signifie que l'entrepôt contient 2 camions de pompiers, 3 poupées Parpie, 2 jeux de construction Lebo…

Cette représentation n'étant pas pratique, le père Noël souhaite convertir ces liste en un dictionnaire qui associe à chaque jouet sa quantité dans l'entrepôt. Par exemple, à partir de `liste_1`, il obtient `dico_1` :

```
1 liste_1 = ['Camion de pompiers', 'poupée Parpie', 'Jeu de construction Lebo','poupée Parpie', 'poupée Parpie', 'Manga Houane Pice', 'Écharpe bleue','Jeu de société SkyDassin', 'puzzle Kopémon', 'Jeu de société SkyDassin','figurine heroman', 'Jeu vidéo Delza', 'puzzle Kopémon', 'écharpe verte','Jeu de construction Lebo', 'Camion de pompiers', 'puzzle Kopémon']
```

```
2 dico_1 = {'Camion de pompiers': 2, 'poupée Parpie': 3, 'Jeu de construction Lebo': 2,'Manga Houane Pice': 1, 'Écharpe bleue': 1, 'Jeu de société SkyDassin': 2,'puzzle Kopémon': 3, 'figurine heroman': 1, 'Jeu vidéo Delza': 1,'écharpe verte': 1}
```

Q1. Écrire une fonction de signature `liste_vers_dico(liste: list[str]) -> dict[str: int]` qui prend en entrée la liste `liste` et renvoie le dictionnaire correspondant. On ferra en sorte de ne parcourir `liste` qu'**une seule fois**.

Correction Q1.

```
1 def liste_vers_dico(liste):
2   dico = {}
3   for el in liste:
4     if el in dico :
5       dico[el] += 1
6     else :
7       dico[el] = 1
8   return dico
```

À partir de maintenant, le stock de jouets est représenté par le dictionnaire `stock: dict[str: int]` décrit ci-dessus.

Q2. Écrire une fonction de signature `compter_jouets(stock: dict[str: int]) -> int` qui renvoie le nombre de jouets dans l'entrepôt du père Noël. Par exemple, `compterJouets(dico_1)` vaut 17.

Correction Q2.

```
1 def compter_jouets(stock):
2   nb_j = 0
3   for jouet in stock.keys():
4     nb_j += stock[jouet]
5   return nb_j
```

Q3. Écrire une fonction de signature `max_jouets(stock: dict[str: int]) -> list[str]` qui renvoie la liste des jouets les plus présents dans l'entrepôt.

Correction Q3.

```
1  def max_jouets(stock):
2    max = 0
3    liste_max = []
4    for e in stock.keys():
5      if stock[e] > max :
6        max = stock[e]
7        liste_max = [e]
8      elif stock[e] == max :
9        liste_max.append(e)
10   return liste_max
```

Par exemple pour `dico_1`, ce sont les poupées Parpie et les puzzles Kopémon qui sont les plus présents (3 exemplaires de chaque). Ainsi, `maxJouets(dico_1)` s'évalue en ["poupée Parpie", "puzzle Kopémon"].

Chaque jour, les lutins du père Noël construisent de nouveaux jouets, ce qui nécessite une mise à jour régulière du stock. Pour garder un historique, le père Noël fait une copie journalière du dictionnaire avant sa mise à jour.

Q4. Écrire une fonction de signature `copie_dico(dico: dict[str: int]) -> dict[str: int]` qui renvoie une copie du dictionnaire `dico` à l'aide d'une **boucle for**.

Correction Q4.

```
1 # remarque : on pourrait faire copie = dico.copy()
2 def copie_dico(dico):
3   copie = {}
4   for e in dico.keys():
5     copie[e] = dico[e]
6   return copie
```

À la fin de la journée, chaque lutin enregistre les jouets qu'il a construit dans un dictionnaire. Par exemple :
si `constructions = {'Jeu de construction Lebo': 150, 'Livre Henry Botteur': 200}`, cela signifie que le lutin a construit 150 jeux de construction Lebo et 200 livres Henry Botteur.

Les jouets construits par l'ensemble des lutins sont consignés dans une liste `liste_const: list[dict[str: int]]` dont chaque élément est le dictionnaire associé à l'un des lutins. Étant donnés le dictionnaire `stock` et la liste `liste_const`, le père Noël souhaite mettre à jour son stock.

Par exemple, si le stock initial est `dico_1` et que les lutins ont construit les jouets suivants :

```
1 liste_const_1 = [
2   {'Jeu de construction Lebo': 150, 'Livre Henry Botteur': 200},
3   {'Manga Houane Pice': 500},
4   {'Camion de pompiers': 100, 'poupée Parpie': 100, 'Jeu de construction Lebo': 100}
5 ]
```

alors le stock mis à jour sera :

```
1 stock_2 = {'Camion de pompiers': 102, 'poupée Parpie': 103, 'Jeu de construction
  Lebo': 252,'Manga Houane Pice': 501, 'Écharpe bleue': 1, 'Jeu de société
  SkyDassin': 2, 'puzzle Kopémon': 3, 'figurine heroman': 1, 'Jeu vidéo Delza': 1,
  'écharpe verte': 1, 'Livre Henry Botteur': 200}
```

Q5. Écrire une fonction
`ajout_prod(stock: dict[str: int], liste_const_1: list[dict[str: int]]) -> dict[str: int]`
qui ajoute la production des lutins dans le stock.
La fonction commencera par créer une copie de `stock`, puis modifiera cette copie (sans modifier `stock`).

Correction Q5.

```python
def ajout_prod(stock, liste_const):
  nouv_stock = copie_dico(stock) # on créé un nouveau stock

  # liste_const est une liste de dictionnaire, on parcourt donc les dictionnaires
  for dico_const in liste_const :
    # pour chaque dictionnaire on vient ajouter les nouveaux jouets créés
    for jouet in dico_const :
      if jouet in nouv_stock :
        nouv_stock[jouet] += dico_const[jouet]
      else :
        nouv_stock[jouet] = dico_const[jouet]
  return nouv_stock
```

2. Réorganisation de l'entrepôt

On note `jouets` la liste sans doublon contenant les noms de tous les jouets présents dans l'entrepôt du père Noël. Par exemple, si le stock est décrit par le dictionnaire `stock_2`, alors la liste correspondante est la liste `jouets_1` suivante.

```
jouets_1 = ['Camion de pompiers', 'poupée Parpie', 'Jeu de construction Lebo',
'Manga Houane Pice', 'Écharpe bleue', 'Jeu de société SkyDassin', 'puzzle Kopémon',
'figurine heroman', 'Jeu vidéo Delza', 'écharpe verte', 'Livre Henry Botteur']
```

Q6. À l'aide d'une **compréhension** de liste, écrire une fonction `stock_to_j(stock: dict[str: int]) -> list[str]` qui renvoie la liste `jouets` décrite ci-dessus. La liste renvoyée ne doit pas contenir de doublon et votre programme ne doit être composé que d'une unique compréhension de liste.

Correction Q6.

```
def stock_to_j(stock):
  return [cle for cle in stock.keys()]
```

Dans cette partie, on appelle ***caractère spécial*** un caractère ne faisant pas partie des 52 lettres minuscules et majuscules de l'alphabet (« a », « b », . . ., « z », « A », « B », . . ., « Z »).

Par exemple, les signes de ponctuation (« « , « ! », « . », . . .), les chiffres (« 0 », « 1 », . . ., « 9 »), les lettres accentuées (« é », « à », « É », . . .) sont des caractères spéciaux.

En Python, on pourra utiliser les fonctions `ord(c: str) -> int` et `chr(i: int) -> str` qui établissent une correspondance entre les chaînes de caractères de taille 1 et les entiers. Par exemple :

s	"A"	"B"	"C"	...	"X"	"Y"	"Z"	"a"	"b"	"c"	...	"x"	"y"	"z"
ord(s)	65	66	67	...	88	89	90	97	98	99	...	120	121	122

i	65	66	67	...	88	89	90	97	98	99	...	120	121	122
chr(i)	"A"	"B"	"C"	...	"X"	"Y"	"Z"	"a"	"b"	"c"	...	"x"	"y"	"z"

s	" "	"!"	"."	"0"	"1"	...	"9"	"é"	"à"	"É"
ord(s)	32	33	46	48	49	...	57	233	224	201

i	32	33	46	48	49	...	57	233	224	201
chr(i)	" "	"!"	"."	"0"	"1"	...	"9"	"é"	"à"	"É"

Dans la suite, on dira qu'une liste `jouets: list[str]` est **standardisée** si toutes ses chaînes de caractères sont **non vides** et ont pour **premier caractère une lettre** majuscule ou bien un caractère spécial. En d'autres termes, les noms commençant par l'une des 26 lettres minuscules sont interdits.

Afin de standardiser la nomenclature de son stock, le père Noël modifie la liste `jouets` obtenue par la fonction `stock_to_j` : lorsque le nom d'un jouet commence par une lettre minuscule, il la remplace par sa version majuscule.

Q7. Écrire une fonction `str_vide(liste: list[str]) -> bool` qui renvoie `True` si la chaîne de caractères vide appartient à `liste` et `False` sinon.

Correction Q7.

```
1 def str_vide(liste):
2   for el in liste:
3     if el == "":
4       return True
5   return False
```

Q8. Écrire une fonction `standardisation(jouets: list[str]) -> list[str]` qui prend en entrée une liste de chaînes de caractères et modifie chaque chaîne de caractères commençant par une lettre minuscule en remplaçant cette dernière par la lettre majuscule correspondante. Votre fonction déclenchera une erreur si `jouets` contient une chaîne de caractères vide. Vous devrez utiliser le tableau ci-dessus pour le décalage.

Correction Q8.

```
1  def standardisation(jouets):
2    assert not str_vide(jouets) # on renvoie une erreur si on a un élément vide
3
4    difference = ord("a") - ord("A") # on peut stocker la différence entre le
     minuscule et le majuscule
5    for i in range(len(jouets)):
6      prems = ord(jouets[i][0])
7      if prems >= ord("a") and prems <= ord("z"): # lettre minuscule
8        # on récupère la lettre majuscule correspondante grace à chr()
9        nouvelle_lettre = chr(prems - difference)
10       # on modifie le nom du jouet à l'indice i en prenant la nouvelle première
         lettre puis le reste du str
11       jouets[i] = nouvelle_lettre + jouets[i][1:]
12
13   return jouets
```

On suppose maintenant que la liste `jouets` est standardisée. Afin de gagner en productivité, le père Noël souhaite trier son stock de jouets par ordre alphanumérique (c'est à dire par ordre alphabétique en autorisant les caractères spéciaux). Il confie cette tache à 27 lutins numérotés de 0 à 26. Chacun des 26 premiers lutins est associé à une lettre de l'alphabet et va gérer les jouets dont le nom commence par cette lettre. Le 27ème lutin va gérer les jouets dont le nom commence par un caractère spécial.

Rappel sur les notations des ensembles :

- `ens = set()` crée un ensemble vide
- `ens.add(e)` ajoute l'élément `e` à l'ensemble `ens`
- `ens.remove(e)` supprime l'élément `e` à l'ensemble `ens`

Q9. Écrire une fonction `repartition(jouets: list[str]) -> list[set[str]]` qui renvoie une liste `rep` de taille 27 telle que `rep[i]` est l'**ensemble** (type `set`) des jouets attribués au lutin numéro `i`. On pourra supposer sans le vérifier que `jouets` est bien standardisée.

Correction Q9.

```python
def repartition(jouets):
    rep =[set() for _ in range(27)] # liste de 27 ensembles

    for j in jouets :
        ord_prem = ord(j[0])
        if ord_prem < ord("A") or ord_prem > ord("Z") : # caractère special
            i = 26 # pour le lutin à caractère spécial
        else :
            i = ord_prem - ord("A") # on récupère le numéro du lutin associé
        # on ajoute dans l'ensemble (qui ne pas pas ajouter de doublons)
        rep[i].add(j)

    return rep
```

Maintenant que chacun des 27 lutins est en charge d'une partie de l'entrepôt, sa mission est de trier les jouets qui lui sont attribués.

Tout d'abord, il faut être capable de comparer deux chaînes de caractères suivant l'ordre alphanumérique. Pour comparer deux chaînes de caractères `s1` et `s2`, on utilise le même principe que pour l'ordre alphabétique d'un dictionnaire papier. En Python, on considérera que le caractère `c1` est avant `c2` dans l'ordre alphanumérique lorsque `ord(c1)` $\leqslant$`ord(c2)`.

Remarque : en Python, on pourrait comparer `s1` et `s2` directement avec les opérateurs booléens habituels ($<$, $<=$, $==$, . . .), mais on se l'interdit dans cette question. Les seules comparaisons autorisées sont les comparaisons entre deux entiers `ord(c1)` et `ord(c2)` où `c1` et `c2` sont des caractères.

Par exemple :

s1	"A"	"B"	""	""	"abcd"	"abc"	"abed"
s2	"b"	"a"	""	"abcd"	""	"abcd"	"abcd"
comp(s1, s2)	-1	-1	0	-1	1	-1	1

Q10. Écrire une fonction `comp(s1: str, s2: str) -> int` qui renvoie :

- `-1` si `s1` est strictement avant `s2` dans l'ordre alphanumérique
- `0` si `s1` et `s2` sont égales
- `1` si `s1` est strictement après `s2` dans l'ordre alphanumérique

Correction Q10.

```python
def comp(s1, s2):
  taille_min = min([len(s1),len(s2)]) # on récupère la plus petite liste
  # on parcours les str jusqu'à la taille du plus petit s
  for i in range(taille_min):
      if ord(s1[i]) < ord(s2[i]):
          return -1
      elif ord(s1[i]) > ord(s2[i]):
          return 1
  # si on n'a pas return -1 ou 1 c'est que les taille_min premiers caractères sont les mêmes, on va devoir donc regarder la taille
  if len(s1) == len(s2):
      return 0
  elif len(s1) < len(s2):
      return -1
  else :
      return 1
```

Étant donnée une liste `liste : list[str]` dont les éléments sont distincts deux à deux, on souhaite construire une liste `liste_triee` ayant les mêmes éléments que `liste` et triée par ordre alphanumérique.

Pour cela, on va utiliser le **tri par insertion** dont voici le pseudo-code :

Initialement, la liste `liste_triee` est vide.
Pour chaque élément `e` de `liste` :
- Si tous les éléments de `liste_triee` sont avant `e` dans l'ordre alphanumérique :
  - i est len(`liste_triee`)
  - on insert `e` à l'indice i de `liste_triee`
- Sinon:
  - i est défini comme le plus petit indice tel que `comp(liste_triee[i], e) == 1`.
  - on insert `e` à l'indice i de `liste_triee`

Voici l'évolution de ces différentes variables lors du tri de la liste
`liste = ["Abc", "Az", "Abb", "Abcd","Abd"]`

	e	i	liste_triee
Initialisation			`[]`
fin du 1er tour	`"Abc"`	0	`["Abc"]`
fin du 2ème tour	`"Az"`	1	`["Abc", "Az"]`
fin du 3ème tour	`"Abb"`	0	`["Abb", "Abc", "Az"]`
fin du 4ème tour	`"Abcd"`	2	`["Abb", "Abc", "Abcd", "Az"]`
fin du 5ème tour	`"Abd"`	2	`["Abb", "Abc", "Abd", "Abcd", "Az"]`

Q11. À l'aide d'une boucle **while**, écrire une fonction de signature `indice_insert(liste_triee: list[str], e: str)->int` qui renvoie l'entier `i` comme défini dans le pseudo-code. `i` est donc l'indice auquel on viendra insérer `e`, on ne demande pas de l'insérer tout de suite.

Correction Q11.

```
def indice_insert(liste_triee, e):
    if len(liste_triee) == 0:
        return 0
    if comp(liste_triee[-1], e) != 1 : # cas ou e est après le dernier élement
        return len(liste_triee)
    i = 0
    while comp(liste_triee[i], e) != 1 :
        i += 1
    return i
```

Q12. En supposant que `e` contient au plus 100 caractères, quelle est la complexité de la fonction `indice_insert` ? Justifier.

Correction Q12. La complexité en pire cas de la fonction `indice_insert` est $O(n)$ où n est la taille de `liste_triee`. En effet au maximum on ne parcourt qu'une fois la liste par indice grâce à `i`.

Q13. Écrire une fonction `insert(liste: list[str], e: str, i: int) -> list[str]` qui insère l'élément `e` dans `liste_triee` à l'indice `i`.
Si la condition $0 \leqslant$ `i` $\leqslant$ `|liste|` n'est pas respectée, votre fonction déclenchera une erreur.

Correction Q13.

```
def insert(liste, e, i):
    assert i >= 0 and i <= len(liste)
    return liste[:i] + [e] + liste[i:]
```

Q14. Quelle est la complexité de la fonction `insert` ? Justifier.

Correction Q14. La compléxité en pire cas de la fonction `insert` est $O(n)$ où n est la taille de la liste, en effet on fait 2 slices un de taille `i` et un de taille `len(liste)-i`. Celà revient donc à un parcourt de liste.

Q15. À l'aide des questions précédentes, écrire une fonction de signature `tri_insertion(liste: list[str]) -> list[str]` qui renvoie la liste triée obtenue en appliquant un tri par insertion sur `liste`.

Correction Q15.

```python
def tri_insertion(liste):
    liste_triee = []
    for e in liste :
        i = indice_insert(liste_triee, e)
        liste_triee = insert(liste_triee, e, i)
    return liste_triee
```

Q16. En supposant que chaque élément de `liste` contient au plus 100 caractères, quelle est la complexité de la fonction `tri_insertion` ? Justifier.

Correction Q16. On pose n la taile de la liste `liste`. On a une boucle for qui parcourt la liste, on regarde maintenant ce qu'on a dans la boucle. On a l'appelle à la fonction `indice_insert` qui est en $O(n)$ (car au liste_triee fait la même taille que liste à la fin), et de même pour l'appelle à la fonction `insert`. On a donc une complexité en $O(n * n) = O(n^2)$.

Dans le but d'améliorer le temps d'exécution de la fonction `tri_insertion`, on propose de réécrire l'étape de recherche de `i` l'aide d'une recherche dichotomique.

Q17. Écrire une fonction de signature
`indice_insert_dicho(liste_triee: list[str], e: str) -> int` qui renvoie l'entier `i` comme défini dans le pseudocode à l'aide d'une recherche dichotomique. On garantira une complexité logarithmique en la taille de `liste_triee`.

Correction Q17.

```python
def indice_insert_dicho(liste_triee, e):
    debut = 0
    fin = len(liste_triee)-1
    while debut <= fin :
        milieu = (fin + debut) // 2
        c = comp(liste_triee[milieu], e)
        if c == 0:
            return milieu
        elif c == 1:
            fin = milieu - 1
        else:
            debut = milieu + 1
    return debut
```

Q18. On appelle `tri_insertion_dicho` la fonction obtenue en prenant le code de `tri_insertion` et en y remplaçant les appels à `indice_insert` par des appels à `indice_insert_dicho`. En supposant que chaque élément de `liste` contient au plus 100 caractères, quelle est la complexité de la fonction `tri_insertion_dicho` ? Justifier.

Correction Q18. Comme on a toujours un appelle à la fonction insertion qui elle est en $O(n)$, même si `indice_insert_dicho` est en $O(\log(n))$ notre complexité reste donc en $O(n^2)$.

3. Lettres au père Noël

Soit $n \in \mathbb{N}$ un entier. Pour s'entraîner à gérer les lettres des enfants, le père Noël demande à sa femme, la mère Noël, de générer toutes les combinaisons possibles de jouets.

Ainsi, étant donnée une liste de jouets `jouets: list[str]` sans doublon, la mère Noël doit créer une liste de listes `comb: list[list[str]]` dont les sous-listes sont de taille n et représentent toutes les combinaisons possibles de n jouets.

Attention, si deux listes contiennent les mêmes jouets (éventuellement dans un ordre différent), alors elles sont considérées comme identiques ; une seule de ces deux listes doit donc apparaître dans `comb`. De plus, chaque jouet peut apparaître au plus une fois dans une sous-liste de `comb`.

Q19. Écrire une fonction de signature
`make_comb(jouets: list[str], n: int) -> list[list[str]]` qui construit la liste `comb`.

Correction Q19.

```
1  # On vient créer une liste qui contient toutes les combinaisons puis on filtre
   pour garder que les combinaisons de tailles n
2  def make_comb(jouets, n):
3    comb = [[]] # dans comb on stock toutes les combinaisons possibles
4
5    for jouet in jouets:
6        # pour chaque jouet on ajoute ce jouet à toutes les combinaisons existantes
7        nouveaux = []
8        for c in comb:
9            c2 = c.copy()
10           c2.append(jouet)
11           nouveaux.append(c2)
12       comb += nouveaux
13
14   # on renvoie seulement les combinaisons de taille n
15   return [c for c in comb if len(c) == n]
```

Les lutins ont lus toutes les lettres des enfants et ont répertorié toutes les données dans un fichier tableur `csv` (coma seperated values) où chaque colonne est séparé par une virgule. Chaque enfant à le droit à 3 cadeaux. Le fichier contient les 4 colones suivante :

- Nom Prénom
- Cadeau 1
- Cadeau 2
- Cadeau 3

Voici les 3 premières lignes du fichier :

Nom Prénom	Cadeau 1	Cadeau 2	Cadeau 3
CHARLES Paul	poupée Parpie	cartes Kopémon	Jeu vidéo Delza
THOMAS Camille	Camion de pompiers	Jeu vidéo Delza	ONE

Q20. Écrire une fonction de signature `liste_enfants_cadeaux(nom_fichier: str) -> dict[str : list[str]]` qui prend en argument le chemin du fichier et renvoie un dictionnaire dans lequel les clés sont les « Nom Prénom » de l'enfant et les valeurs la liste des cadeaux qui leurs sont associés.

Correction Q20.

```python
def liste_enfants_cadeaux(nom_fichier):
    # lecture des lignes du fichier
    fichier = open(nom_fichier)
    enfants = fichier.readlines()
    fichier.close()

    dict_enfants = {}
    for i in range(1, len(enfants)):
        donnee_enfant = enfants[i].strip().split(',')
        # on pourrait gérer le cas ou 2 enfants ont le même nom/prénom
        if donnee_enfant[0] not in dict_enfants :
            dict_enfants[donnee_enfant[0]] = donnee_enfant[1:]
    return dict_enfants
```

4. Optimisation de la tournée

Le jour de Noël, le père Noël ne peut pas prendre tous les jouets dans sa hotte en une seule fois, mais souhaite tout de même la remplir le plus possible.

Formellement, on associe à la hotte une capacité $C \in \mathbb{N}$ et à chaque jouet un entier naturel représentant le volume qu'il occupe. Ainsi, si on note $k \in \mathbb{N}$ le nombre total de jouets, on dispose d'une liste triée `volumes = [v0, v1, . . . , vk-1]` où `vi` $\in \mathbb{N}$ est le volume du jouet numéro `i`. Le père Noël souhaite donc choisir un sous-ensemble $I \subset [\![0, k-1]\!]$ tel que :

$$A(I) : \sum_{i \in I} v_i \leq C$$

La stratégie du père Noël est dite optimale si pour tout $C \in \mathbb{N}$ et toute liste `volumes` , cette stratégie lui permet de construire un ensemble I vérifiant $A(I)$ et tel que pour tout $J \subset [\![0, k-1]\!]$ vérifiant $A(J)$, on ait :

$$\sum_{j \in J} v_j \leq \sum_{i \in I} v_i$$

En d'autres termes, la stratégie est **optimale** si elle permet de **remplir la hotte au maximum sans dépasser la capacité C**.

Stratégie gloutonne 1. La première stratégie consiste à emporter en priorité le jouet avec **le plus petit volume**. Le père Noël emporte donc les jouets 0, 1, 2, . . . et s'arrête lorsque l'objet suivant ne rentre plus dans la hotte.

Par exemple, avec C = 7 et `volumes = [3, 3, 5, 6]`, on obtient $I = \{0, 1\}$. En effet, les objets d'indices 0 et 1 prennent un volume total de $3 + 3 = 6$. Même si la hotte n'est pas complètement remplie ($6 <$ C), il n'est plus possible d'y ajouter d'objet.

Q21. À l'aide d'une boucle **while**, écrire une fonction de signature `strat1(c: int, volumes: list[int]) -> list[int]` qui renvoie une liste contenant les éléments de l'ensemble I obtenu avec la stratégie décrite ci-dessus. On rappelle que la liste `volumes` est supposée triée.

Correction Q21.

```python
def strat1(c, volumes):
  ens_i = []
  i = 0
  while i < len(volumes) and volumes[i] <= c:
      ens_i.append(i)
      c = c - volumes[i]
      i += 1
  return ens_i
```

Q22. Cette stratégie est-elle optimale ? Justifier en donnant un contre-exemple si ce n'est pas le cas.

Correction Q22.
Cette stratégie n'est pas optimale. En effet, `strat1(4, [1,2,3])` renvoie `[0,1]`, alors qu'une stratégie optimale doit renvoyer `[0,2]`.

Stratégie gloutonne 2. La deuxième stratégie consiste à emporter en priorité le jouet avec le plus grand volume. Le père Noël s'intéresse donc au jouet $n-1$, puis $n-2, ..., 2, 1, 0$. Pour chacun de ces jouets, s'il reste suffisamment de place dans sa hotte, le père Noël emporte le jouet, sinon il passe au jouet suivant.

Par exemple, avec $C = 14$ et `volumes = [1, 2, 2, 5, 7, 7, 8]`, on obtient $I = \{0, 3, 6\}$.

Q23. Écrire une fonction de signature `strat2(c: int, volumes: list[int]) -> list[int]` qui renvoie une liste contenant les éléments de l'ensemble I obtenu avec la stratégie décrite ci-dessus. On rappelle que la liste `volumes` est supposée triée.

Correction Q23.

```python
def strat2(c, volumes):
  ens_i = []
  # on parcourt la liste à l'envers
  for i in range(len(volumes)-1, -1, -1):
      if volumes[i] <= c:
          c = c - volumes[i]
          ens_i.append(i)
  return ens_i
```

Q24. Cette stratégie est-elle optimale ? Justifier en donnant un contre-exemple si ce n'est pas le cas.

Correction Q24.
Cette stratégie n'est pas optimale. En effet, `strat2(5, [2,3,4])` renvoie `[2]`, alors qu'une stratégie optimale doit renvoyer `[0,1]`.

Q25. Bonus : Proposer une fonction `strat_opt(c: int, volumes: list[int]) -> list[int]` qui renvoie une liste contenant les éléments de l'ensemble I obtenu avec une stratégie optimale. Justifier votre réponse.

Correction Q25.

```python
def strat_opt(c, volumes):
    n = len(volumes)

    # on construit une matrice de taille c*n où
    # dp[i][j] = volume max atteignable avec les i premiers objets et capacité j
    # on va remplir la matrice
    dp = [[0 for _ in range(c+1)] for _ in range(n+1)]

    for i in range(1, n+1):
        for j in range(c+1):
            if volumes[i-1] <= j:
                dp[i][j] = max(
                    dp[i-1][j],
                    dp[i-1][j-volumes[i-1]] + volumes[i-1]
                )
            else:
                dp[i][j] = dp[i-1][j]

    # reconstruction de la solution grace à la matrice
    res = []
    j = c
    for i in range(n, 0, -1):
        if dp[i][j] != dp[i-1][j]:
            res.append(i-1)
            j -= volumes[i-1]

    return res[::-1]
```